

Entendendo o arquivo server.js

Explicação passo a passo de Node.js, Express, middlewares, CORS, arquivos estáticos e MySQL

Material de apoio para alunos iniciantes

Este guia explica a base do servidor apresentada no arquivo server.docx. Os termos técnicos são mantidos porque fazem parte da profissão, mas aparecem acompanhados de significado simples, exemplos e alertas.

Ideia central: o server.js é o ponto de entrada do back-end. Ele prepara a aplicação, configura os recursos necessários e coloca o servidor para escutar requisições.

1. O que é o server.js?

server.js é apenas um nome de arquivo. Ele não possui poderes especiais no Node.js. O projeto o utiliza como arquivo principal porque nele ficam as configurações do servidor e, frequentemente, as rotas da API.

Parte do projeto	Responsabilidade
Front-end	Mostra a interface e interage com o usuário: HTML, CSS e JavaScript do navegador.
Back-end	Recebe requisições, aplica regras, acessa o banco e envia respostas.
server.js	Inicializa e configura o back-end deste projeto.
MySQL	Armazena os produtos de forma permanente.
Pasta public	Contém os arquivos que podem ser entregues diretamente ao navegador.

Analogia: o server.js funciona como a preparação de uma loja antes de abrir: instala os equipamentos, define onde estão os produtos, conecta o sistema ao estoque e abre a porta em um endereço.

2. O que acontece ao executar node server.js?

1. O sistema operacional inicia o programa Node.js.
2. O Node.js lê e executa o arquivo server.js de cima para baixo.
3. As bibliotecas são carregadas com require.
4. A aplicação Express é criada.
5. Os middlewares são registrados na ordem em que aparecem.
6. A conexão com o MySQL é configurada e testada.
7. app.listen abre a porta 3000 para receber requisições.
8. O processo continua ativo, aguardando eventos como conexões e consultas ao banco.

Importante: Node.js é o ambiente que executa JavaScript fora do navegador. Express é uma biblioteca executada pelo Node.js; eles não são a mesma coisa.

3. Vocabulário essencial

Termo	Explicação simples
Servidor	Programa que fica disponível para receber pedidos e enviar respostas.
Cliente	Programa que faz o pedido, como navegador, Insomnia ou front-end.
Back-end	Parte executada no servidor, fora do navegador do usuário.
Runtime	Ambiente que executa um programa. Node.js é um runtime de JavaScript.
Biblioteca	Código pronto que oferece funcionalidades reutilizáveis.
Framework	Estrutura que organiza a aplicação e orienta como construí-la. Express é frequentemente chamado de framework web minimalista.
Dependência	Pacote externo que o projeto precisa para funcionar.
Módulo	Arquivo ou pacote que exporta funcionalidades para outro código usar.
Middleware	Função executada durante o caminho da requisição antes da resposta final.
Callback	Função que será chamada posteriormente, quando uma operação terminar.
Assíncrono	Trabalho que termina depois, sem obrigar o programa inteiro a ficar parado.
Porta	Número que identifica um serviço dentro de um computador.
localhost	Nome que representa o próprio computador em uso.

4. Pacotes e instalação

O código completo usa três dependências externas:

Pacote	Função no projeto
express	Cria o servidor web, middlewares e rotas.
mysql2	Permite que o Node.js se comunique com o MySQL.
cors	Adiciona cabeçalhos que controlam acessos feitos por outras origens.

```
npm install express mysql2 cors
```

- npm consulta o registro de pacotes e instala as dependências.
- package.json registra informações do projeto e suas dependências.
- package-lock.json registra versões exatas para instalações reproduzíveis.
- node_modules armazena localmente os pacotes instalados e seus próprios pacotes dependentes.

Dúvida comum: não se deve escrever manualmente o conteúdo de node_modules. Essa pasta é administrada pelo npm.

5. Importação das bibliotecas

```
const express = require("express");
```

require solicita ao sistema de módulos do Node.js que carregue o pacote express. O valor exportado pelo pacote é guardado na constante express.

```
const mysql = require("mysql2");
```

O pacote mysql2 fornece funções para criar conexões e executar comandos SQL no MySQL.

```
const cors = require("cors");
```

O pacote cors fornece um middleware que configura cabeçalhos relacionados à política de mesma origem do navegador.

Termo técnico - CommonJS: require e module.exports fazem parte do sistema de módulos CommonJS. Projetos modernos também podem usar import e export, chamados de ES Modules, mas não se deve misturar os dois formatos sem configurar o projeto.

Por que const? const impede que a variável seja apontada para outro valor depois. Isso não significa que o objeto interno seja totalmente imutável.

6. Criação da aplicação Express

```
const app = express();
```

express é uma função. Ao executá-la com parênteses, criamos o objeto da aplicação e o guardamos em app. Esse objeto permite registrar middlewares, rotas e iniciar o servidor.

Exemplo	Uso
app.use(...)	Registra um middleware.
app.get(...)	Registra uma rota GET.
app.post(...)	Registra uma rota POST.
app.listen(...)	Inicia a escuta em uma porta.

Aplicação e servidor: no uso cotidiano dizemos que app é o servidor. Tecnicamente, app representa a aplicação Express; app.listen cria o servidor HTTP que passa a escutar a porta.

7. O que é middleware?

Middleware significa algo que trabalha no meio do caminho. No Express, uma requisição passa por uma sequência de funções. Cada função pode ler ou modificar req e res, encerrar a resposta ou permitir que o processamento continue.

```
Cliente -> middleware 1 -> middleware 2 -> rota -> resposta
```

A ordem importa: `app.use` deve normalmente aparecer antes das rotas que dependem daquele middleware. Caso contrário, a rota poderá receber `req.body` vazio ou não receber os cabeçalhos esperados.

8. Recebendo JSON

```
app.use(express.json());
```

`express.json` cria um middleware que interpreta corpos enviados como JSON. Depois da leitura, os dados ficam disponíveis em `req.body`.

```
{
  "nome": "Mouse Gamer",
  "categoria": "Periféricos"
}
```

Após o middleware interpretar esse conteúdo, uma rota pode acessar `req.body.nome` e `req.body.categoria`.

Cabeçalho necessário: o cliente normalmente deve enviar `Content-Type: application/json`. Esse cabeçalho informa o formato do corpo da requisição.

Limite conceitual: interpretar JSON não é o mesmo que validar JSON. `express.json` consegue ler os campos, mas a aplicação ainda precisa verificar se eles existem e se possuem valores aceitáveis.

9. CORS

```
app.use(cors());
```

`cors` cria um middleware que adiciona cabeçalhos HTTP permitindo acessos entre origens conforme sua configuração.

Uma origem é formada principalmente por protocolo, domínio e porta. Mudar qualquer um desses componentes pode criar outra origem.

Endereço	Origem
<code>http://localhost:3000</code>	HTTP + localhost + porta 3000
<code>http://localhost:5500</code>	Outra origem, pois a porta mudou
<code>https://localhost:3000</code>	Outra origem, pois o protocolo mudou

Por que o navegador bloqueia? A Same-Origin Policy é uma proteção do navegador contra páginas acessando recursos de outras origens sem autorização. CORS é o mecanismo pelo qual o servidor informa quais acessos aceita.

Segurança: `app.use(cors())` sem opções é amplo e conveniente para aula. Em produção, costuma-se restringir as origens autorizadas. CORS não substitui login, autorização nem validação.

10. Entregando a pasta public

```
app.use(express.static("public"));
```

`express.static` cria um middleware de arquivos estáticos. Ele procura os arquivos solicitados dentro da pasta `public` e os entrega sem precisar criar uma rota para cada arquivo.

Arquivo no projeto	Endereço no navegador
public/index.html	http://localhost:3000/
public/style.css	http://localhost:3000/style.css
public/script.js	http://localhost:3000/script.js
public/imagens/produto.jpg	http://localhost:3000/imagens/produto.jpg

Arquivo estático: é um arquivo entregue praticamente como está armazenado, como HTML, CSS, JavaScript do navegador e imagens. Isso é diferente de uma rota de API que executa lógica e gera uma resposta.

Por que index.html abre em /? servidores de arquivos estáticos normalmente tratam `index.html` como o documento padrão de uma pasta.

11. Iniciando o servidor

```
app.listen(3000, function(){
  console.log("Servidor rodando em http://localhost:3000");
});
```

`app.listen` solicita que o servidor escute a porta 3000. O callback é executado quando a escuta foi iniciada. `console.log` apenas mostra uma mensagem no terminal; ele não inicia o servidor.

Parte	Significado
3000	Porta escolhida para esta aplicação.
localhost	O próprio computador.
http://	Protocolo usado na comunicação.
callback	Confirma que o servidor começou a escutar.

Porta ocupada: se outro programa já estiver usando a porta 3000, o servidor poderá falhar com um erro semelhante a `EADDRINUSE`.

Acesso externo: `localhost` só representa a máquina atual. Para outro computador acessar o projeto, são necessários endereço de rede, configuração adequada e liberação no firewall.

12. Conexão com o MySQL

```
const db = mysql.createConnection({
```

`createConnection` cria um objeto de conexão. O objeto precisa saber onde está o MySQL e quais credenciais utilizar.

Configuração	Significado
host: "localhost"	O MySQL está no mesmo computador do Node.js.
user: "root"	Usuário utilizado para autenticação no MySQL.
password: "123456"	Senha do usuário informado.
database: "atividade_single_page"	Banco selecionado para as consultas.

Host e banco não são a mesma coisa: host indica onde o servidor MySQL está; database indica qual banco será usado dentro desse servidor.

Segurança de credenciais: a senha escrita diretamente no código é aceitável apenas neste exercício local. Em projetos reais, segredos devem ficar em variáveis de ambiente e nunca ser publicados no GitHub.

Exemplo conceitual para um projeto real:

```
password: process.env.DB_PASSWORD
```

`process.env` permite ler variáveis fornecidas pelo ambiente. Ainda seria necessário configurar `DB_PASSWORD` fora do código.

13. Testando a conexão

```
db.connect(function(erro){
```

`db.connect` tenta abrir a comunicação com o MySQL. A operação é assíncrona; quando terminar, o `callback` será chamado.

```
if(erro){
  console.log("Erro ao conectar ao banco:", erro);
  return;
}
```

Se algo falhar, erro conterá informações técnicas. A mensagem e o objeto são exibidos no terminal. `return` encerra o `callback` para que a mensagem de sucesso não seja executada.

```
console.log("Conectado ao banco de dados MySQL!");
```

Essa linha só é alcançada quando não existe erro na tentativa de conexão.

Possível falha	O que conferir
Access denied	Usuário e senha.

Unknown database	Nome do banco e se ele foi criado.
ECONNREFUSED	Se o serviço MySQL está iniciado e a porta está correta.
Tabela inexistente	Nome da tabela e banco selecionado.

Não mostre detalhes ao usuário: erros completos do banco ajudam no terminal durante o desenvolvimento, mas não devem ser enviados diretamente ao cliente, pois podem revelar estrutura, credenciais ou informações internas.

14. Assincronismo e ordem real dos acontecimentos

O Node.js executa as chamadas de cima para baixo, mas não espera toda operação assíncrona terminar para avançar.

```
db.connect(function(erro){ ... });
app.listen(3000, function(){ ... });
```

Primeiro, o programa inicia a tentativa de conexão. Em seguida, registra a escuta da porta. Os callbacks serão executados quando cada operação terminar. Portanto, a mensagem do servidor pode aparecer antes da confirmação do banco.

Consequência: o servidor pode começar a receber requisições antes de a conexão com o banco ter sido confirmada. Para uma aula inicial, a estrutura é suficiente; sistemas reais costumam controlar melhor a inicialização e a recuperação de falhas.

Outro detalhe: o return dentro do callback de db.connect encerra somente aquele callback. Ele não fecha automaticamente o servidor nem encerra todo o processo Node.js.

15. Leitura do código completo em blocos

Ordem	Bloco	Responsabilidade
1	require	Carregar Express, MySQL2 e CORS.
2	express()	Criar a aplicação.
3	express.json	Interpretar corpos JSON.
4	cors	Configurar acesso entre origens.
5	express.static	Entregar a pasta public.
6	createConnection	Preparar as configurações do MySQL.
7	db.connect	Testar a conexão.
8	app.listen	Abrir a porta 3000.
9	rotas	Normalmente ficam antes de app.listen e usam app e db.

Onde entram as rotas? depois dos middlewares e da criação de db, mas normalmente antes de app.listen. JavaScript precisa conhecer app e db antes que as rotas possam utilizá-los.

16. Código completo comentado

```
const express = require("express");
const mysql = require("mysql2");
const cors = require("cors");

const app = express();

app.use(express.json());
app.use(cors());
app.use(express.static("public"));

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "123456",
  database: "atividade_single_page"
});

db.connect(function(erro){
  if(erro){
    console.log("Erro ao conectar ao banco:", erro);
    return;
  }

  console.log("Conectado ao banco de dados MySQL!");
});

app.listen(3000, function(){
  console.log("Servidor rodando em http://localhost:3000");
});
```

17. Como testar

9. Confirme que o MySQL está iniciado.
10. Confirme que o banco atividade_single_page existe.
11. Confira usuário e senha no objeto de conexão.
12. Abra o terminal na pasta que contém server.js e package.json.
13. Execute node server.js.
14. Observe as mensagens do terminal.
15. Acesse <http://localhost:3000> para testar a pasta public.
16. Depois, utilize navegador ou Insomnia para testar as rotas da API.

O terminal precisa permanecer aberto: o processo Node.js fica ativo enquanto o servidor está funcionando. Encerrar o processo interrompe o servidor.

18. Dúvidas comuns

Pergunta	Resposta
Express substitui o Node.js?	Não. Express é executado pelo Node.js.

require instala a biblioteca?	Não. npm install instala; require carrega uma biblioteca já disponível.
console.log inicia o servidor?	Não. Quem inicia é app.listen.
localhost é a internet?	Não. É o próprio computador.
A porta 3000 é obrigatória?	Não. É uma escolha comum; outra porta livre poderia ser usada.
express.json cria JSON?	Ele interpreta o JSON recebido. res.json é usado para enviar JSON.
CORS protege a API inteira?	Não. Ele controla política de origem no navegador, não autenticação ou permissões.
public significa arquivo público na internet?	Significa que o Express pode entregá-lo diretamente a quem acessar o servidor.
db.connect cria o banco?	Não. Ele conecta a um banco que deve existir.
return fecha o Node.js?	Não. Nesse código, apenas encerra a função callback atual.

19. Modelo mental para montar o servidor

17. Instale e carregue as dependências.
18. Crie a aplicação Express.
19. Registre os middlewares gerais antes das rotas.
20. Configure o acesso aos arquivos estáticos, se necessário.
21. Crie a conexão com o banco sem expor credenciais em projetos reais.
22. Trate a confirmação ou falha da conexão.
23. Crie as rotas que utilizarão app e db.
24. Inicie a escuta em uma porta disponível.
25. Teste separadamente front-end, servidor, banco e rotas.

Resumo final: Node.js executa o arquivo; require carrega pacotes; Express cria a aplicação; middlewares preparam as requisições; mysql2 conversa com o banco; app.listen abre a porta para os clientes.